Programmation Web en PHP

Informatique Appliquée - S5

Pr. Ilham SLIMANI

i.slimani@ump.ac.ma

2025/2026

Chapitre 8

Gestion des formulaires

I. Introduction

Les formulaires constituent le principal moyen pour vos visiteurs d'entrer des informations sur votre site. Ils constituent donc l'élément essentiel qui permet l'interactivité entre un site et ses visiteurs.

Tout échange entre visiteur et serveur passe par un formulaire, dans lequel l'utilisateur peut saisir **textes** ou **mots de passe**, opérer des **choix** grâce à des boutons **radio**, des **cases à cocher** ou des **listes** de sélection, voire envoyer ses propres **fichiers** depuis le poste client.

Le langage HTML ne peut pas analyser les informations que le visiteur a saisi dans un formulaire. Alors, le traitement des résultats doit s'effectuer dans un autre langage, par exemple le **PHP**.

II. Création d'un formulaire HTML

1. Structure d'un formulaire HTML

```
<!DOCTYPE html>
<html>
	<head>
		<meta charset="UTF-8">
		<title>Titre de la page</title>
	</head>
	<body>
	<form method="POST" action="<?=$_SERVER['PHP_SELF']?>">
		<fieldset><legend>Titre du formulaire</legend>
			<!--Corps du formulaire-->
		</fieldset>
	</form>
	</body>
</html>
```

2. Attributs de l'élément <form>

`method`="**POST|GET**" : indique la méthode de transmission HTTP des données saisies dans le formulaire.

- «**POST**» est la valeur qui correspond à un envoi de données stockées dans le corps de la requête (les données sont cachées).
- «**GET**» correspond à un envoi des données ***visibles dans l'URL*** (séparées de l'adresse du script par un point d'interrogation) : ***à éviter pour des raisons évidentes de sécurité***.

La méthode **GET** est la ***méthode par défaut*** et présente deux inconvénients:

1) L'ajout des données du formulaire à l'adresse URL du fichier qui les traite, ce qui les rend visibles par le visiteur.
2) Il existe une limite à la longueur des URL (2 à 4ko)

Ces problèmes ne se retrouvent pas avec la valeur POST, que vous utiliserez dans la plupart des cas.

2. Attributs de l'élément <form>

`action`="**nom_de_fichier.php**" permet d'indiquer l'**URL** qui va recevoir les informations entrées dans le formulaire, lorsque l'on cliquera sur le bouton de validation. Plus précisément, l'URL est l'adresse d'un programme (un script) qui va récupérer les données et les traiter.

Si le fichier qui traite les données est celui qui contient le formulaire vous pouvez utiliser les syntaxes suivants:

```
1) action="<?=$_SERVER['PHP_SELF']?>"
2) action="#"
3) action=""
```

L'attribut `action` peut aussi avoir la valeur "mailto:", qui provoque l'envoi des données vers une adresse e-mail, qu'il faut préciser à la suite du mot mailto en écrivant, par exemple : `action="mailto: nom@gmail.com"`

3. Attributs optionnels de l'élément <form>

`name`="**chaine_de_caractères**" Attribue un nom au formulaire. Cet attribut est surtout utilisé pour accéder aux éléments du formulaire via un script JavaScript.

`enctype`="**type_d'encodage**" **:** C'est le type d'encodage des données transmises au serveur.

1) "**application/x-www-form-urlencoded**" **:** c'est la valeur par défaut. C'est celle qu'il faut utiliser dans la plupart des cas.
2) "**multipart/form-data**" **:** C'est la valeur à utiliser pour envoyer des fichiers.

***Remarque*:** Si l'attribut action a la valeur "`mailto`:", l'attribut **enctype** a pour valeur "text/plain" ou "text/html", selon que le contenu envoyé à une adresse e-mail au format texte ou XHTML.

4. Ecrire un formulaire

La balise **<input>** définit des champs d'information. L'attribut **name** associe un nom au champ, l'attribut **value** désigne éventuellement une valeur initiale, et l'attribut **type** décrit le type de champ de formulaire. Les valeurs possibles incluent:

- ✓ **`text`** : champ de texte qui permet à l'utilisateur de saisir un nombre ou une phrase.
- ✓ **`checkbox`** : cases à cocher.
- ✓ **`radio`** **:** boutons radio.
- ✓ **`password`** :pour un champ de mot de passe.
- ✓ **`hidden`** : champ caché pour transmettre des informations vers le serveur, sans que l'utilisateur ne le sache.

4. Ecrire un formulaire

- ✓ **textarea** pour une zone de texte multi-lignes : `<textarea> </textarea>`
- ✓ **select** :pour une liste de sélection (ou liste déroulante).
- ✓ **submit** : pour un bouton de soumission.
- ✓ **`image`** : même principe que submit mais c'est une image au lieu d'un bouton.
- ✓ **`file`** : Affiche un bouton permettant de sélectionner un fichier de l'ordinateur.
- ✓ **`email,url,number,date,etc`**. : pour des types de données spécifiques.
- ✓ **`reset`** :pour remettre le formulaire dans son état initial.

Remarque :

Les **checkbox** et **radios** n'envoient rien s'ils ne sont pas cochés.

Exemple

Créer une page php (**formulaire.php**) contenant un formulaire avec les champs suivants:

- Vos coordonnées qui contient:
  - Nom (zone de texte), Prénom (zone de texte),Mail (zone de texte).
  - Sexe (boutons radio): Homme ou Femme.
  - Ville (liste déroulante).

- Vos loisirs
  - Loisirs (cases à cocher): Lecture, Sport, Voyage.
  - Zone de texte multilignes pour décrire vos loisirs.

- Envoyer nous votre photo
  - Champ d'envoi d'un fichier du client vers le serveur.
  - Champ caché contenant la taille maximale des fichiers transférables.
  - Boutons de réinitialisation.
  - Bouton d'envoi.

L’interface du formulaire

Vos coordonnées

Nom :

Prénom :

Mail :

Sexe : ○ Homme ○ Femme

Ville Paris

Vos loisirs

Loisirs : ☐ Voyage ☐ Lecture ☐ Sport

Commentaire : Décrivez vos loisires

Envoyer nous votre photo

Choisir un fichier Aucun fichier choisi

Envoyer Effacer

III. Récupération des données du formulaire

1. valeurs uniques

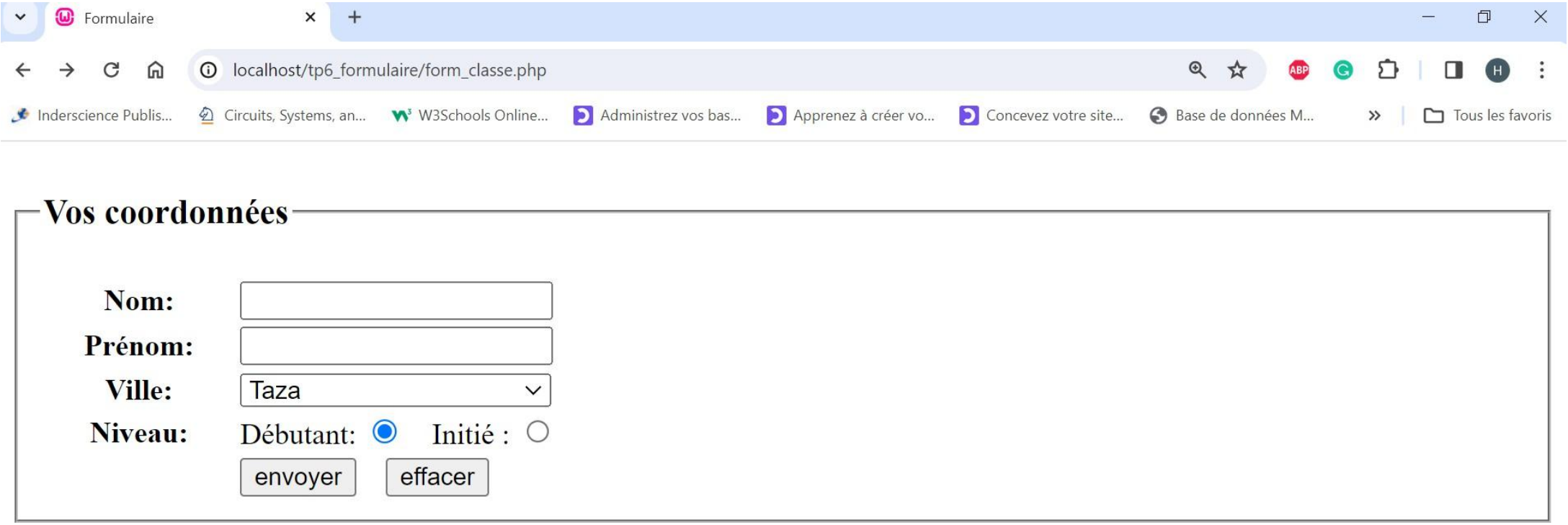
- Les valeurs uniques proviennent des champs de formulaire dans lesquels l'utilisateur ne peut entrer qu'une valeur, un texte par exemple, ou ne peut faire qu'un seul choix (*bouton radio, liste de sélection à choix unique*).

- Selon la méthode choisie, ces valeurs sont contenues sur le serveur dans des tableaux associatifs **$_POST** et **$_GET**. Les clés de ces tableaux sont les noms associés aux champs par l'attribut **name**.

Cas de la méthode POST

Exemple 1: Récupération des valeurs dans un formulaire élémentaire

1) Ecrire un code HTML qui permet d'avoir le formulaire de la figure suivante :



2) Ecrire un code PHP qui permet de récupérer le résultat dans le même fichier, comme suit :

Cas de la méthode GET

Exemple 2: On remplace la méthode POST par la méthode GET.

formulaire élémentaire

localhost/Formulaire/formulaire2.php?Nom=+Zaaraoui&Prenom=Hicham&niveau=Initié&ok=envoyer

Infos

Nom: Zaaraoui

Prénom: Hicham

Débutant : ○ **Initié :** ◉

envoyer effacer

Récupération des données

Bonjour **Zaaraoui Hicham** tu es **Initié** en php

Remarque:

Lors du clic sur le bouton d'envoi l'adresse de la page cible désignée par l'attribut **action** est suivie par le caractère ? puis par le nom de chaque champ et la valeur qui y est associée.

2. Les valeurs multiples

- Cela concerne un groupe de ***cases à cocher*** ayant le même attribut `name` ou ***une liste de sélection*** ayant toujours un nom unique mais dans laquelle l'attribut `multiple` est défini.

Exemple:

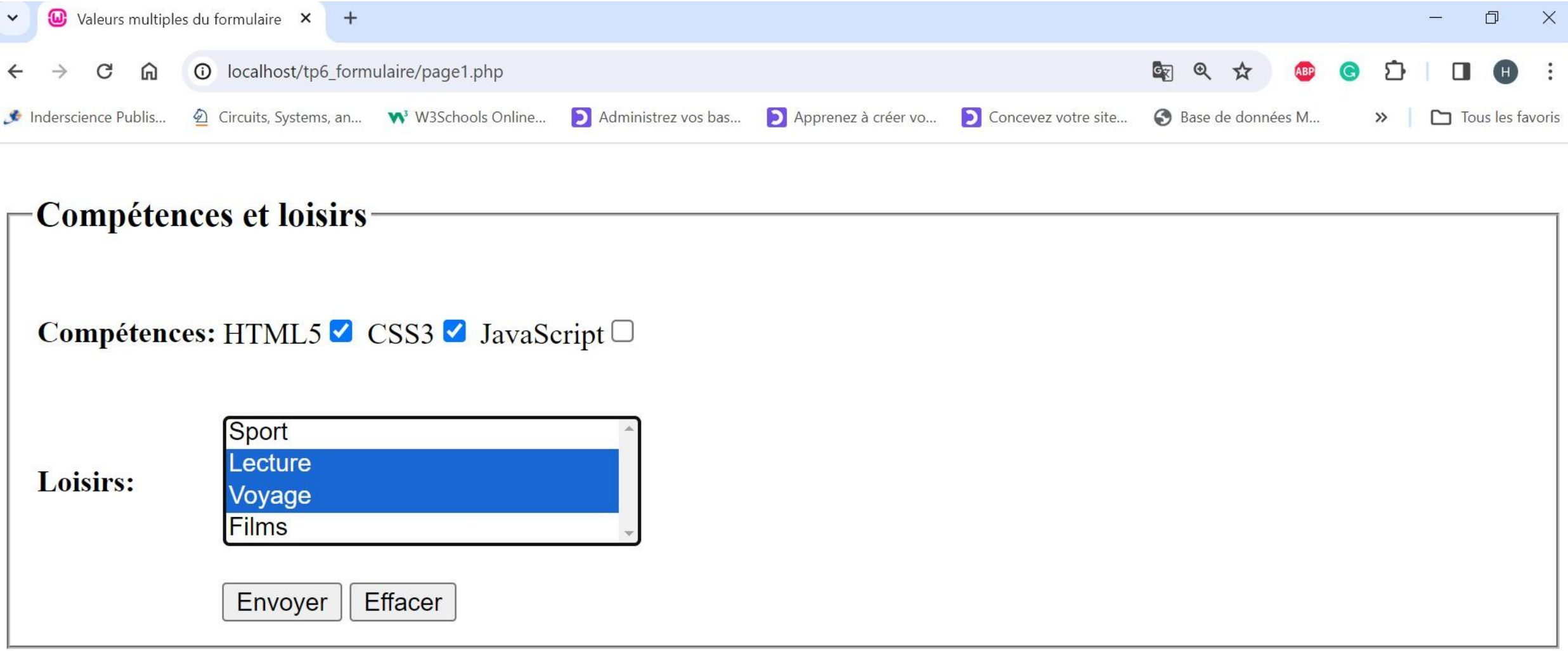
Bleu : `<input type="checkbox" name="choix[]" value="bleu" />`

Rouge: `<input type="checkbox" name="choix[]" value="rouge" />`

- Pour récupérer ces valeurs on utilise:
  - ✓ `$_POST["choix"][0]` qui contient la valeur **`bleu`**.
  - ✓ `$_POST["choix"][1]` qui contient la valeur **`rouge`**.

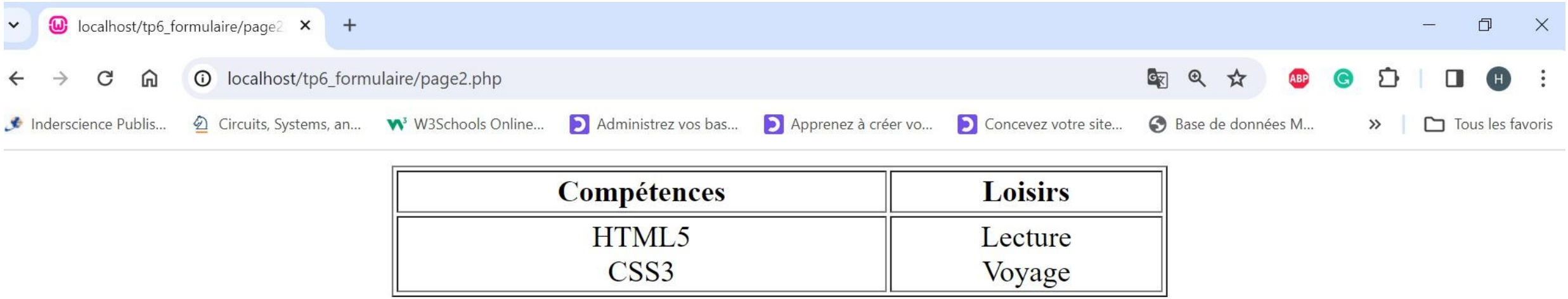
Exemple 3 : Récupération des valeurs multiple du formulaire

1) Ecrire une page **page1.php** qui permet d'afficher le formulaire suivant:



2) Ecrire le script de la page **page2.php** qui permet d'afficher l'interface suivante :

3) Ecrire un autre code dans la page **page3.php** qui permet d'afficher l'interface suivante :



1) page1.php Solution

```
<body>
<fieldset> <legend><h3>Compétences et loisirs</h3></legend>
<table>
<form method='post' action="page2.php">
<tr> <th>Compétences:</th>
<td> HTML5<input type="checkbox" name="Competences[]" value="HTML5"/>
CSS3<input type="checkbox" name="Competences[]" value="CSS3"/>
JavaScript<input type="checkbox" name="Competences[]" value="JavaScript"/>
</td> </tr>
<tr> <th>Loisirs:</th> <td> <select name="Loisirs[]" multiple>
<option value="Sport">Sport</option>
<option value="Lecture">Lecture</option>
<option value="Voyage">Voyage</option>
<option value="Films">Films</option>
</select> </td> </tr>
<tr> <th colspan="2" class="al"> <input type="submit" value="Envoyer"
name="ok"/> <input type="reset" value="Effacer" name="no"/> </th> </tr>
</form> </table>
</fieldset> </body>
```

2) page2.php Solution

```
<meta charset="UTF-8"/>
<?php
if(isset($_POST['ok'])){
    $ch="Compétences : ";
        foreach($_POST['Competences'] as $val){
            $ch.="<b>".$val." </b>";
        }
$ch.="<br>";
$ch.="Loisirs : ";
        foreach($_POST['Loisirs'] as $v){
            $ch.="<b>".$v." </b>";
        }
echo $ch;
}
?>
```

3) page3.php Solution

```php
<?php
if(isset($_POST['ok'])){
    $ch="<table width='50%' align='center' border='1'>";
$ch.="<tr><th>Compétences</th><th>Loisirs</th></tr>";
$ch.="<tr align='center'><td>";
    foreach($_POST['Competences'] as $val){
        $ch.=$val."<br>";
    }
$ch.="</td><td>";
     foreach($_POST['Loisirs'] as $v){
        $ch.=$v."<br>";
     }
$ch.="</td></tr></table>";
echo $ch; }
?>
```

IV. Maintien de l'état du formulaire

- Après le traitement, le formulaire se retrouve alors dans son état initial, et toutes les saisies effectuées sont effacées.

- En cas d'erreur de saisie sur un seul champ, l'utilisateur est obligé de recommencer l'ensemble de la saisie.

Pour éviter cela,

- Pour ***le texte***, il suffit de définir l'attribut **value** avec la variable $_POST[name] ou $_GET[name]:

Exemple:

```
value="<?php if(isset($_POST["nom"])) echo $_POST["nom"]?>"
```

- Pour ***les boutons radio***, il faut définir l'attribut du bouton choisi à la valeur "**checked**" en fonction de la valeur de la variable `$_POST[name]` (ou `$_GET[name]`).

Exemple:

```
<?php if(isset($_POST["niveau"])&& $_POST["niveau"]=="debutant")
echo "checked"; ?>
```

- Pour ***les listes déroulante***, il faut définir l'attribut du bouton choisi à la valeur "**selected**" en fonction de la valeur de la variable `$_POST[name]` (ou `$_GET[name]`).

Exemple:

```
<?php if(isset($_POST["ville"])&&$_POST["ville"]=="Paris")
echo "selected"; ?>
```